



INSTITUTO POLITÉCNICO NACIONAL

Unidad Interdisciplinaria de
Ingeniería y Ciencias Sociales y
Administrativas



ABSTRACCIÓN Y USO DE DATOS

Secuencia: 3CM30

Integrantes:

Gomez Rodriguez Aldo Sebastian - NL13

Nieto Ramirez Jean - NL 31

Nuñez Solis Luis Angel - NL 32

Link de Repositorio:

<https://github.com/LuisxD14/Abstraccion>

Reporte de Prácticas

1. Secuencia de Fibonacci

Este programa genera la sucesión matemática de Fibonacci hasta el número de términos que el usuario solicite. Es un ejercicio clásico para comprender ciclos e iteraciones sucesivas donde cada número es la suma de los dos anteriores.

El programa define una función obtenerFibonacci() que primero solicita al usuario la cantidad de términos deseados. Incluye validaciones fundamentales: verifica que el número ingresado sea mayor a 0 y que no exceda 93. Esta última validación es crítica porque a partir del término 94, el valor supera la capacidad máxima de un tipo de dato unsigned long long en C++, lo que causaría un desbordamiento de memoria (overflow).

```
Menu principal
1 Fibonacci
2 Matrices
3 Triangulo de Sierpinski
4 Polvo de Cantor
5 Salir
Elige una opcion: 1
Cuantos terminos desea mostrar: 5

Menu de formato
1 txt
2 json
3 xml
4 csv
Elige un formato: 1
```

Una vez validado, utiliza un ciclo for tradicional para calcular la secuencia usando variables auxiliares (a, b y c) que van guardando y sumando los términos anteriores. Todo el resultado se va concatenando en un flujo de cadena (ostringstream) para su posterior despliegue y guardado.

```

    string obtenerFibonacci() {
        int n;
        cout << "Cuantos terminos desea mostrar: ";
        cin >> n;

        if (n <= 0) {
            return "Programa Fibonacci\nNo se puede generar una secuencia con esa cantidad de terminos.\n";
        }

        if (n > 93) {
            return "Programa Fibonacci\nLa cantidad solicitada es demasiado grande para este programa.\n";
        }

        ostringstream out;
        out << "Programa Fibonacci\n";
        out << "Terminos solicitados: " << n << "\n";
        out << "Secuencia\n";

        unsigned long long a = 0, b = 1;
        for (int i = 0; i < n; ++i) {
            if (i == 0) {
                out << "F(0) = 0\n";
            } else if (i == 1) {
                out << "F(1) = 1\n";
            } else {
                unsigned long long c = a + b;
                a = b;
                b = c;
                out << "F(" << i << ") = " << b << "\n";
            }
        }
    }

```

2. Operaciones con Matrices

Este módulo permite al usuario realizar operaciones de álgebra lineal básicas pero fundamentales: multiplicar una matriz por un valor escalar (constante) y calcular el producto cruzado de dos matrices diferentes (Matriz A por Matriz B).

```
Menu principal
1 Fibonacci
2 Matrices
3 Triangulo de Sierpinski
4 Polvo de Cantor
5 Salir
Elige una opcion: 2
Filas de la matriz A: 2
Columnas de la matriz A: 2
Filas de la matriz B: 3
Columnas de la matriz B: 3
Constante para multiplicar A: 4
Ingresa los valores de A
A[1][1]: 4
A[1][2]: 5
A[2][1]: 4
A[2][2]: 5
Ingresa los valores de B
B[1][1]: 3
B[1][2]: 4
B[1][3]: 3
B[2][1]: 4
B[2][2]: 5
B[2][3]: 4
B[3][1]: 5
B[3][2]: 6
B[3][3]: 7
```

Al ejecutarse, la función `obtenerMatrices()` pide al usuario las dimensiones (filas y columnas) tanto de la matriz A como de la matriz B, así como un valor constante. Los datos se almacenan de forma dinámica en memoria utilizando `vector<vector<double>>`, lo que permite manejar matrices de cualquier tamaño sin declarar un tamaño estático previo.

El programa incluye funciones modulares como leerMatriz() para poblar los datos, multiplicarConstante() que itera por cada elemento multiplicándolo por el escalar, y multiplicarMatrices(). En esta última, se aplica una validación matemática indispensable: revisa que el número de columnas de A sea exactamente igual al número de filas de B. Si esta regla matemática no se cumple, el programa aborta la operación para evitar errores y muestra un mensaje indicando que las dimensiones no son compatibles. Las operaciones matemáticas se realizan mediante tres ciclos for anidados.

```
string obtenerMatrices() {
    int filasA, columnasA, filasB, columnasB;
    double constante;

    cout << "Filas de la matriz A: ";
    cin >> filasA;
    cout << "Columnas de la matriz A: ";
    cin >> columnasA;
    cout << "Filas de la matriz B: ";
    cin >> filasB;
    cout << "Columnas de la matriz B: ";
    cin >> columnasB;
    cout << "Constante para multiplicar A: ";
    cin >> constante;

    if (filasA ≤ 0 || columnasA ≤ 0 || filasB ≤ 0 || columnasB ≤ 0) {
        return "Programa Matrices\nLas dimensiones deben ser mayores que cero.\n";
    }

    vector<vector<double>> A = leerMatriz(filasA, columnasA, "A");
    vector<vector<double>> B = leerMatriz(filasB, columnasB, "B");

    ostringstream out;
    out << "Programa Matrices\n";
    out << "Matriz A\n";
    out << formatearMatriz(A, "");
    out << "Matriz B\n";
    out << formatearMatriz(B, "");

    out << "Constante\n";
    out << constante << "\n";

    vector<vector<double>> Aconstante = multiplicarConstante(A, constante);
    out << "Resultado de A por constante\n";
    out << formatearMatriz(Aconstante, "");
```

3. Triángulo de Sierpinski

Genera una representación visual en consola del Triángulo de Sierpinski, un fractal matemático famoso que se forma subdividiendo un triángulo en triángulos más pequeños de manera recursiva. El usuario debe ingresar el "nivel" del fractal, con una validación que restringe el nivel a un máximo de 8 para evitar que la consola colapse por el exceso de caracteres. A nivel de código, se reserva un espacio de dibujo o "lienzo" bidimensional utilizando un vector de strings lleno de espacios en blanco.

```
Menu principal
1 Fibonacci
2 Matrices
3 Triangulo de Sierpinski
4 Polvo de Cantor
5 Salir
Elige una opcion: 3
Nivel del fractal: 3
```

```
Triangulo de Sierpinski
      *
     * *
    *  *
   * * *
  *   *
 * *   *
* * * *
* * * *
 * * * *
  * * * *
    * * * *
     * * * *
      * * * *
```

La magia ocurre en la función `sierpinskiFill()`, la cual es completamente recursiva. Recibe las coordenadas iniciales y el tamaño, y si el tamaño es 1, dibuja un asterisco (*). Si no, calcula la mitad del tamaño y se llama a sí misma tres veces para dibujar los tres vértices que forman el triángulo. Este es un excelente ejemplo de la estrategia algorítmica "divide y vencerás".

```
void sierpinskiFill(vector<string>& g, int row, int col, int size) {
    if (size == 1) {
        g[row][col] = '*';
        return;
    }
    int half = size / 2;
    sierpinskiFill(g, row, col, half);
    sierpinskiFill(g, row + half, col - half, half);
    sierpinskiFill(g, row + half, col + half, half);
}
```

```

string obtenerSierpinski() {
    int nivel;
    cout << "Nivel del fractal: ";
    cin >> nivel;

    if (nivel < 0 || nivel > 8) {
        return "Triangulo de Sierpinski\nEl nivel debe estar entre 0 y 8.\n";
    }

    int altura = 1 << nivel;
    int ancho = altura * 2 - 1;
    vector<string> g(altura, string(ancho, ' '));
    sierpinskiFill(g, 0, ancho / 2, altura);

    ostringstream out;
    out << "Triangulo de Sierpinski\n";
    for (const auto& fila : g) {
        out << fila << "\n";
    }
    return out.str();
}

```

4. Polvo de Cantor

Dibuja una representación del Polvo de Cantor (adaptado a un plano bidimensional), otro fractal geométrico que se construye dividiendo un segmento en tres partes iguales y eliminando el tercio central de manera iterativa.

```
Menu principal
1 Fibonacci
2 Matrices
3 Triangulo de Sierpinski
4 Polvo de Cantor
5 Salir
Elige una opcion: 4
Nivel del polvo de Cantor: 3
```

```
Polvo de Cantor
#####
#  ##  ##  ##  ##  ##  ##  ##  #
#####
###      #####      #####      ###
#  #    #  ##  #    #  ##  #    #  #
###      #####      #####      ###
#####
#  ##  ##  ##  ##  ##  ##  ##  #
#####
#####
#  ##  ##  #          #  ##  ##  #
#####
#####
###      #####      #####      ###
#  #    #  ##  #    #  ##  #    #  #
###      #####      #####      ###
#####
#  ##  ##  #          #  ##  ##  #
#####
#####
#  ##  ##  ##  ##  ##  ##  ##  #
#####
#####
###      #####      #####      ###
#  #    #  ##  #    #  ##  #    #  #
###      #####      #####      ###
#####
#  ##  ##  ##  ##  ##  ##  ##  #
#####
```


La función obtenerCantor() restringe el nivel máximo a 5, ya que el tamaño del lienzo crece en potencias de 3 (3^{nivel}). Se prepara un "lienzo" de espacios en blanco del tamaño calculado y se manda a llamar a la función recursiva cantorFill().

En lugar de dibujar con líneas, esta función utiliza el carácter #. Funciona dividiendo la sección actual en una cuadrícula de 3x3. Recorre esta cuadrícula usando dos ciclos for y realiza una llamada recursiva en cada celda, excepto en la celda central (coordenadas 1, 1), la cual se salta con la instrucción continue, dejando así los característicos "huecos" del fractal de Cantor.

```
void cantorFill(vector<string>& g, int row, int col, int size) {
    if (size == 1) {
        g[row][col] = '#';
        return;
    }
    int part = size / 3;
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            if (i == 1 && j == 1) continue;
            cantorFill(g, row + i * part, col + j * part, part);
        }
    }
}

string obtenerCantor() {
    int nivel;
    cout << "Nivel del polvo de Cantor: ";
    cin >> nivel;

    if (nivel < 0 || nivel > 5) {
        return "Polvo de Cantor\nEl nivel debe estar entre 0 y 5.\n";
    }

    int tam = 1;
    for (int i = 0; i < nivel; ++i) tam *= 3;

    vector<string> g(tam, string(tam, ' '));
    cantorFill(g, 0, 0, tam);

    ostringstream out;
    out << "Polvo de Cantor\n";
    for (const auto& fila : g) {
        out << fila << "\n";
    }
    return out.str();
}
```

5. Sistema de Exportación de Archivos

A diferencia de los programas anteriores este código permite exportar el resultado de cualquier programa anterior y guardarlo en el disco duro en múltiples formatos estándar (TXT, JSON, XML y CSV).

El corazón de esta funcionalidad está en el menú principal (main) y la función guardarArchivo(). Una vez que uno de los 4 programas anteriores se ejecuta, su salida no solo se muestra en consola, sino que se almacena en una variable de tipo string. Luego, se solicita al usuario el formato de salida.

Para garantizar que los archivos generados sean válidos, el programa implementa funciones de "Escape" (xmlEscape, jsonEscape, csvEscape). Estas funciones recorren la cadena de resultados buscando caracteres que puedan romper la sintaxis del formato elegido (como comillas dobles en JSON o signos de "mayor que" en XML) y los reemplazan por sus equivalentes seguros. Finalmente, utiliza la librería <fstream> para crear un archivo físico en la misma carpeta donde se ejecuta el programa e inyecta la información formateada correctamente.

```
void guardarArchivo(const string& programa, const string& extension, const string& contenido) {
    string nombre = programa + "." + extension;

    ofstream archivo(nombre);
    if (!archivo) {
        cout << "No se pudo crear el archivo\n";
        return;
    }

    if (extension == "txt") {
        archivo << contenido;
    } else if (extension == "json") {
        archivo << "{\n";
        archivo << "  \"programa\": \"" << jsonEscape(programa) << "\",\n";
        archivo << "  \"contenido\": \"" << jsonEscape(contenido) << "\"\n";
        archivo << "}\n";
    } else if (extension == "xml") {
        archivo << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
        archivo << "<archivo>\n";
        archivo << "  <programa>" << xmlEscape(programa) << "</programa>\n";
        archivo << "  <contenido><![CDATA[" << contenido << "]]></contenido>\n";
        archivo << "</archivo>\n";
    } else if (extension == "csv") {
        archivo << "programa,contenido\n";
        string limpio = contenido;
        for (char& c : limpio) {
            if (c == '\n') c = ' ';
        }
        archivo << csvEscape(programa) << "," << csvEscape(limpio) << "\n";
    }

    cout << "Archivo guardado como " << nombre << "\n";
}
```

6. Sistema de Gestión de Dígrafos (Grafo Dirigido)

Este programa implementa una estructura de datos basada en grafos dirigidos (dígrafos) mediante Programación Orientada a Objetos. Sirve para almacenar, gestionar y visualizar conexiones unidireccionales entre distintos puntos (nodos). Es ideal para modelar problemas del mundo real donde la dirección importa, como rutas de transporte de un solo sentido, redes de distribución o flujos de trabajo, permitiendo registrar el tiempo y costo exacto de cada trayecto.

```
1. Mostrar Digrafo
2. Agregar conexion
3. Guardar archivo
4. Salir
Opcion: 1
Nodos: { }
Aristas: { }
Origen      Destino      Arista      Tiempo      Costo
1. Mostrar Digrafo
2. Agregar conexion
3. Guardar archivo
4. Salir
Opcion: 2
Origen: 34
Destino: 35
Arista: 23
Tiempo: 23
Costo: 1790
Ruta agregada.
1. Mostrar Digrafo
2. Agregar conexion
3. Guardar archivo
4. Salir
Opcion: 1
Nodos: { 34 35 }
Aristas: { 23 }
Origen      Destino      Arista      Tiempo      Costo
34          35          23          23          1790
```

El programa utiliza dos clases principales: `ConexionNodo2Nodo` y `Digrafo`. La primera encapsula los datos de una sola ruta (nodo inicial u origen, nodo final o destino, nombre de la arista, tiempo y costo).

La segunda administra estas conexiones utilizando la librería estándar de C++ (STL): emplea la estructura set para mantener un registro de nodos y aristas únicas sin duplicados, y un vector para las rutas dinámicas. Al ejecutarse, el programa intenta cargar datos previos desde un archivo de texto (digrafo_datos.txt) simulando la persistencia de datos. Mediante un menú interactivo en consola, el usuario puede visualizar la lista de adyacencia del dígrafo, agregar nuevas rutas (con validación para evitar tiempos o costos negativos) y guardar los cambios. Las funciones de lectura y escritura están diseñadas para parsear líneas de texto separadas por comas, convirtiendo secuencias de caracteres nuevamente en objetos estructurados.

```
class Digrafo {
private:
    set<string> nodos;
    set<string> aristas;
    vector<ConexionNodo2Nodo> rutas;

public:
    void agregarRuta(const string& init, const string& fin, const string& arista, double t, double c) {
        if (t < 0 || c < 0) {
            cout << "Error: Datos invalidos." << endl;
            return;
        }

        ConexionNodo2Nodo nuevaConexion(init, fin, arista, t, c);
        rutas.push_back(nuevaConexion);

        nodos.insert(init);
        nodos.insert(fin);
        aristas.insert(arista);

        cout << "Ruta agregada." << endl;
    }

    void mostrarDigrafo() const {
        cout << "Nodos: { ";
        for (const auto& n : nodos) cout << n << " ";
        cout << "}" << endl;

        cout << "Aristas: { ";
        for (const auto& a : aristas) cout << a << " ";
        cout << "}" << endl;

        cout << left << setw(12) << "Origen" << setw(12) << "Destino"
            << setw(15) << "Arista" << setw(10) << "Tiempo" << setw(10) << "Costo" << endl;

        for (const auto& r : rutas) {
            cout << left << setw(12) << r.getNodoInicial()
                << setw(12) << r.getNodoFinal()
                << setw(15) << r.getAristaConexion()
                << setw(10) << r.getTiempo()
            << endl;
        }
    }
};
```

Estructuras de Datos, Algoritmos de Ordenamiento y Grafos en C++

Análisis completo de tres módulos: TADs (Pila, Cola, Lista), algoritmos de ordenamiento (Burbuja, Merge Sort, Quick Sort) y estructura de Grafo con exportación a múltiples formatos.

Lenguaje	C++17
Módulos	adt ordenamiento grafo
Patrones	OOP, Herencia, Polimorfismo, Punteros dinámicos
Exportación	JSON · XML · CSV · TXT

1. Módulo ADT — Estructuras de Datos Lineales

El módulo ADT (*Abstract Data Types*) implementa tres estructuras clásicas: **Pila** (LIFO), **Cola** (FIFO) y **Lista** estática. Cada clase encapsula sus datos internos y expone una interfaz pública homogénea, facilitando su uso desde menús interactivos y su posterior serialización a cuatro formatos de archivo.

1.1 Clase Pila

Implementa una pila de enteros con arreglo de 100 posiciones. El puntero de tope es el entero **cantidad**; al apilar se incrementa y al desapilar se decrementa, sin necesidad de mover datos en memoria.

Fragmento clave — agregar / quitar

```
void Pila::agregar(int valor) {
    if (cantidad < 100)
        datos[cantidad++] = valor; // push: escribe y avanza el tope
}

void Pila::quitar() {
    if (cantidad > 0)
        cantidad--; // pop: retrocede el tope (O(1))
    else
        puts("La pila esta vacia.");
}
```

```
}
```

¿Por qué es importante? El comportamiento LIFO es fundamental en sistemas de deshacer/rehacer, llamadas recursivas y evaluación de expresiones. Ambas operaciones son $O(1)$ sin reserva de memoria dinámica.

Serialización — obtenerDatos()

```
vector<string> Pila::obtenerDatos() const {  
    vector<string> resultado;  
    for (int i = cantidad - 1; i >= 0; i--) // de tope a base  
        resultado.push_back(to_string(datos[i]));  
    return resultado;  
}
```

Este método es el puente entre la estructura en memoria y las funciones de exportación. Recorre el arreglo de tope a base, respetando el orden lógico LIFO.

1.2 Clase Cola

Cola circular de capacidad fija $MAX_COLA = 5$. Almacena objetos **Producto** (nombre + precio). Usa índices **inicio** y **fin** con aritmética módulo para reutilizar posiciones liberadas.

Fragmento clave — encolar con buffer circular

```
void Cola::encolar(Producto p) {  
    if (estaLlena()) { cout << "Cola llena\n"; return; }  
    arreglo[fin] = p;  
    fin = (fin + 1) % MAX_COLA; // avance circular  
    cantidad++;  
}  
  
void Cola::desencolar() {  
    if (estaVacia()) { cout << "Cola vacia\n"; return; }  
    inicio = (inicio + 1) % MAX_COLA; // avance circular  
    cantidad--;  
}
```

¿Por qué el buffer circular? Evita desplazar todos los elementos al desencolar (costo $O(n)$), manteniendo ambas operaciones en $O(1)$. Es la base de colas de impresión, schedulers de SO y buffers de red.

1.3 Clase Lista

Lista estática de enteros con capacidad $MAX_LISTA = 10$. Aunque más sencilla que las anteriores, demuestra el patrón general: arreglo + contador, con métodos simétricamente nombrados para consistencia de API.

Fragmento clave — agregar / quitar

```
void Lista::agregarElemento(int valor) {  
    if (cantidad < MAX_LISTA) {  
        datos[cantidad] = valor;  
        cantidad++;  
    } else cout << "Lista llena\n";  
}  
  
void Lista::quitarElemento() { // elimina el ultimo (similar a pila)
```

```

if (cantidad > 0) cantidad--;
else cout << "Lista vacia\n";
}

```

1.4 Funciones de Exportación

Cuatro funciones libres (**convertirAJSON**, **convertirAXML**, **convertirACSV**, **convertirATXT**) aceptan un `vector<string>` genérico, desacoplando el formato de la estructura de datos. Esto sigue el principio de Responsabilidad Única (SRP).

Fragmento — convertirAJSON

```

void convertirAJSON(const string& nombre, const vector<string>& datos) {
    ofstream f(nombre + ".json");
    f << "{\n  \"registros\": [\n";
    for (size_t i = 0; i < datos.size(); ++i) {
        f << " { \"dato\": \"" << datos[i] << "\" }";
        if (i < datos.size()-1) f << ",";
        f << "\n";
    }
    f << " ]\n}\n";
}

```

El patrón se repite para XML, CSV y TXT. La invariante clave es recibir siempre `vector<string>`: cualquier estructura que implemente **obtenerDatos()** puede exportarse sin cambiar las funciones de conversión.

2. Módulo Ordenamiento — Algoritmos Clásicos

El módulo implementa tres algoritmos en clases independientes bajo una jerarquía polimórfica: la clase abstracta **Ordenador** define la interfaz, y **OrdenadorInt**, **OrdenadorChar** y **OrdenadorEstudiante** la especializan para Burbuja. **MergeSortHandler** y **QuickSortHandler** son clases autónomas con sus propias estrategias.

2.1 Clase Abstracta Ordenador

```

class Ordenador {
public:
    virtual void cargar() = 0; // entrada de datos
    virtual void ordenar() = 0; // algoritmo
    virtual void mostrar() = 0; // visualización
    virtual vector<string> obtenerDatos() = 0; // serialización
    virtual ~Ordenador() {}
};

```

¿Por qué una clase abstracta? Permite tratar cualquier ordenador de forma genérica (*Liskov Substitution Principle*). Un puntero `Ordenador*` puede apuntar a `Int`, `Char` o `Estudiante` sin cambiar el código cliente.

2.2 Ordenamiento Burbuja

Algoritmo $O(n^2)$ en tiempo promedio y peor caso. Itera $n-1$ pasadas; en cada una compara pares adyacentes y los intercambia si están fuera de orden. El elemento más grande 'burbujea' hacia el final en

cada pasada.

Implementación — OrdenadorInt::ordenar()

```
void OrdenadorInt::ordenar() {
    for (int i = 0; i < TAM - 1; i++) // n-1 pasadas
        for (int j = 0; j < TAM - 1 - i; j++) // ventana decreciente
            if (datos[j] > datos[j + 1])
                swap(datos[j], datos[j + 1]);
}
```

La optimización clave es la ventana $TAM - 1 - i$: los últimos i elementos ya están en su posición definitiva y no necesitan compararse. Ejemplo con [5, 3, 8, 1]:

Pasada	Estado del arreglo	Intercambios
0	[3, 5, 1, 8]	5↔3, 8↔1
1	[3, 1, 5, 8]	5↔1
2	[1, 3, 5, 8]	3↔1

2.3 Merge Sort

Algoritmo **divide y vencerás** con complejidad $O(n \log n)$ garantizada. Divide recursivamente el arreglo hasta sublistas de un elemento, luego fusiona pares ordenados. La clase usa memoria dinámica temporal en la función merge.

Función merge — fusión de mitades

```
void MergeSortHandler::merge(int izq, int mid, int der) {
    int n1 = mid - izq + 1, n2 = der - mid;
    int* L = new int[n1]; // copia mitad izquierda
    int* R = new int[n2]; // copia mitad derecha
    for (int i = 0; i < n1; i++) L[i] = arr[izq + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i=0, j=0, k=izq;
    while (i<n1 && j<n2) // comparar y fusionar
        arr[k++] = (L[i]<=R[j]) ? L[i++] : R[j++];
    while (i<n1) arr[k++] = L[i++]; // residuo izquierdo
    while (j<n2) arr[k++] = R[j++]; // residuo derecho
    delete[] L; delete[] R; // liberar memoria
}
```

Punto crítico: Los delete[] al final son obligatorios; omitirlos causa fuga de memoria en cada llamada recursiva. Para arreglos grandes, esta función se llama $O(n \log n)$ veces.

Función mergeSort — recursión

```
void MergeSortHandler::mergeSort(int izq, int der) {
    if (izq < der) { // caso base: segmento de 1 elem
        int mid = (izq + der) / 2;
        mergeSort(izq, mid); // mitad izquierda
        mergeSort(mid + 1, der); // mitad derecha
        merge(izq, mid, der); // fusionar resultados
    }
}
```



```
}
```

2.4 Quick Sort

Algoritmo $O(n \log n)$ promedio, $O(n^2)$ peor caso. Opera *in-place* sin copias auxiliares. La clase **QuickSortHandler** almacena objetos **Dato** (letra + número) y permite elegir el criterio de ordenamiento en tiempo de ejecución mediante el campo **criterio**.

Estructura Dato y criterio dinámico

```
struct Dato { char letra; int numero; };

int QuickSortHandler::particion(int bajo, int alto) {
    Dato pivote = lista[alto]; // pivote = ultimo elemento
    int i = bajo - 1;
    for (int j = bajo; j < alto; j++) {
        // criterio elegido por el usuario en tiempo de ejecucion
        bool cond = (criterio==1) ? lista[j].letra < pivote.letra
        : lista[j].numero < pivote.numero;
        if (cond) { i++; intercambiar(lista[i], lista[j]); }
    }
    intercambiar(lista[i+1], lista[alto]);
    return i + 1; // posicion final del pivote
}
```

El uso de **Dato*** (puntero dinámico) en lugar de arreglo fijo permite aceptar cualquier cantidad de elementos ingresada en tiempo de ejecución. El destructor libera esta memoria correctamente.

Comparación de algoritmos

Algoritmo	Mejor caso	Promedio	Peor caso	Espacio extra	Estable
Burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No

3. Módulo Grafo — Estructura y Exportación

El módulo implementa un grafo dirigido y etiquetado compuesto por **Nodos**, **Aristas** y **Rutas** (camino con múltiples conexiones). Toda la lógica reside en un único archivo compilable: *grafo.cpp*.

3.1 Estructuras de Datos

```
struct Nodo { int id; string nombre; };
struct Arista { int id; string nombre; };

struct Conexion { // arco entre dos nodos
    int nodoInicial;
    int nodoFinal;
    int aristaConexion; // id de la arista que los une
}
```

```
};

struct Ruta { // camino = secuencia de conexiones
int id; string nombre;
vector<Conexion> conexiones;
};

struct Grafo { // contenedor principal
vector<Nodo> nodos;
vector<Arista> aristas;
vector<Ruta> rutas;
};
```

El uso de vector permite tamaño dinámico sin límite fijo. La relación Ruta → Conexion → Nodo/Arista modela correctamente un grafo de transporte donde múltiples caminos comparten infraestructura.

3.2 Operaciones CRUD

El menú principal ofrece 15 opciones. Las operaciones de eliminación usan iteradores y erase() de forma idiomática en C++:

Eliminar por ID con iterador

```
void eliminarNodo(Grafo& g) {
int id; cin >> id;
for (auto it = g.nodos.begin(); it != g.nodos.end(); ++it) {
if (it->id == id) {
g.nodos.erase(it); // invalida el iterador -> return
cout << "Nodo eliminado.\n";
return;
}
}
cout << "Nodo no encontrado.\n";
}
```

El **return inmediato** tras erase() es esencial: continuar usando un iterador invalidado es comportamiento indefinido en C++.

3.3 Exportadores de Formato

Cuatro funciones libres exportan el grafo completo. Cada una recibe const Grafo& (referencia constante, sin copia) y escribe su sección con la sintaxis del formato objetivo.

exportJSON — ejemplo de salida generada

```
{
  "nodos": [
    {"id":1, "nombre":"Ciudad A"},
    {"id":2, "nombre":"Ciudad B"}
  ],
  "aristas": [
    {"id":10, "nombre":"Autopista Norte"}
  ],
  "rutas": [
    {"id":100, "nombre":"Ruta Norte", "conexiones":[
```

```
{ "nodoInicial":1, "nodoFinal":2, "aristaConexion":10}
}]
]
```

exportXML — escape de caracteres especiales

```
string xmlEsc(const string& s) {
    string o;
    for (char c : s) {
        if (c == '&') o += "&amp;";
        else if (c == '<') o += "&lt;";
        else if (c == '>') o += "&gt;";
        else o += c;
    }
    return o;
}
```

¿Por qué es importante `xmlEsc`? Un nombre de nodo con '`<`' o '`&`' corrompería el XML produciendo un documento inválido. Esta función sanitiza cualquier string antes de insertarlo entre etiquetas.

3.4 Edición de Rutas — Limpiar y Reconstruir

```
void editarRuta(Grafo& g) {
    // ... localizar ruta por ID ...
    if (ch == 's') {
        r.conexiones.clear(); // vaciar conexiones existentes
        int n; cin >> n;
        for (int i = 0; i < n; i++) {
            Conexion c;
            cin >> c.nodoInicial >> c.nodoFinal >> c.aristaConexion;
            r.conexiones.push_back(c);
        }
    }
}
```

El patrón `clear()` + `push_back()` es más seguro que intentar editar conexiones individuales: garantiza consistencia aunque el usuario cambie el número de conexiones.

4. Archivo grafo_leerG.cpp — Lectura desde Archivo

El archivo complementario **leerG.cpp** contiene la lógica para reconstruir un grafo a partir de un archivo previamente exportado (CSV o TXT). Cierra el ciclo de persistencia: crear → exportar → leer → operar.

El flujo típico es: abrir el archivo con `ifstream`, parsear línea por línea con `getline` e `istringstream`, y reconstruir los vectores de `Nodo`, `Arista` y `Ruta` en el objeto `Grafo`.

5. Patrones de Diseño y Buenas Prácticas

Patron / Practica	Donde se aplica	Beneficio
Encapsulamiento	Pila, Cola, Lista: datos privados + metodos publicos	Protege el estado interno; la interfaz no cambia si cambia la implementacion.
Herencia + Polimorfismo	Ordenador (abstract) -> OrdenadorInt, OrdenadorChar, OrdenadorEstudiante	Codigo cliente generico; anadir un nuevo tipo no requiere cambiar el menu.
SRP (Single Responsibility)	convertirAJSON / XML / CSV / TXT son funciones libres independientes	Cada funcion tiene una sola razon de cambio; facil de probar y mantener.
RAII / Gestion de memoria	QuickSortHandler: new[] en cargar(), delete[] en ~QuickSortHandler()	El destructor garantiza liberacion aunque ocurra una excepcion.
Buffer circular	Cola: indices inicio/fin con modulo MAX_COLA	Encolar y desencolar en $O(1)$ sin desplazar elementos.
Iterador + erase()	eliminarNodo / Arista / Ruta en grafo.cpp	Eliminacion segura respetando la invalidacion de iteradores.

Práctica — Sistema de Calculadora, Manejo de clases(autos) y holamundo

Este conjunto de código implementa un sistema modular que combina manejo de objetos, recursividad y generación dinámica de reportes. El archivo principal (Funciones.h) contiene las definiciones de clases y funciones globales, mientras que el archivo de implementación gestiona los menús y la lógica de reporte.

2.1 Clase Persona

La clase Persona encapsula los datos personales de un individuo. Aplica el principio de encapsulamiento al declarar todos sus atributos como privados y exponerlos únicamente a través de getters y del método mostrar().

Atributos privados:

- nombre, ap, am (apellidos paterno y materno)
- genero (tipo string)
- edad (tipo entero)

Métodos relevantes:

- capturar(): Lee los datos del usuario mediante cin.
- mostrar(): Retorna un string formateado usando stringstream.
- Getters individuales: getNombre(), getAP(), getAM(), getGenero(), getEdad().

2.2 Clase Auto

La clase Auto sigue el mismo patrón de diseño que Persona. Almacena información de un vehículo y expone sus datos mediante una interfaz limpia.

Atributos:

- marca (string)
- precio (double)
- anio (int)

El método mostrar() genera una línea con los tres atributos separados por barras verticales, facilitando su integración en los sistemas de reporte.

2.3 Clase Calculadora

La clase Calculadora implementa cinco operaciones aritméticas básicas (suma, resta, multiplicación, división y potencia) utilizando sobrecarga de métodos y recursividad como estrategias centrales.

Estrategia recursiva:

- Multiplicación: se implementa como sumas repetidas. `multRec(a, b)` devuelve $a + \text{multRec}(a, b-1)$.
- División: se implementa como restas sucesivas. `divRec(a, b)` cuenta cuántas veces cabe b en a .
- Potencia: se calcula como multiplicaciones encadenadas. $\text{potRec}(\text{base}, \text{exp}) = \text{base} * \text{potRec}(\text{base}, \text{exp}-1)$.

Sobrecarga de métodos:

Cada operación cuenta con tres versiones: sin parámetros (retorna neutro), con dos operandos y con tres operandos. Esto permite flexibilidad en el uso sin requerir clases derivadas.

2.4 Sistema de Reportes Multi-Formato

Una de las características más destacadas de esta práctica es la generación de reportes en cuatro formatos distintos para cada tipo de resultado: Hola Mundo, Factorial, datos de Personas/Autos y resultados de Calculadora.

Formatos implementados:

- TXT: salida de texto plano con separadores visuales (====).
- CSV: encabezados con comas, compatible con hojas de cálculo.
- XML: estructura etiquetada con declaración UTF-8.
- JSON: objeto con pares clave-valor y manejo correcto de comas finales.

Cada función de reporte recibe los datos como parámetros y construye su salida utilizando `stringstream`, evitando efectos secundarios y manteniendo la función pura. Esta separación facilita pruebas unitarias y la extensión futura del sistema.

Práctica — Sistema de Arbol con Dijkstra (GrafoPOO)

Esta práctica implementa un sistema completo de gestión de grafos dirigidos con pesos en dos dimensiones (tiempo y costo). El programa permite modelar redes de transporte o distribución, encontrar rutas óptimas y persistir la información en disco en múltiples formatos.

3.1 Estructura ConexionNodo2Nodo

Es una estructura de datos (struct) que agrupa todos los atributos de una arista dirigida en el grafo:

nodoinicial	string — nodo origen de la conexión
nodoFinal	string — nodo destino de la conexión
aristaConexion	string — nombre o etiqueta de la arista
tiempo	double — peso temporal del trayecto
costo	double — peso monetario del trayecto

3.2 Clase GrafoPOO

Es el núcleo del sistema. Administra la colección de rutas y expone las operaciones fundamentales del grafo. Usa tres contenedores internos de la STL de C++:

- `set<string>` nodos: conjunto de nodos únicos, actualizado automáticamente.
- `set<string>` aristas: conjunto de aristas únicas.
- `vector<ConexionNodo2Nodo>` rutas: lista dinámica de todas las conexiones.

Operaciones públicas:

- `ingresarConexion()`: Agrega una nueva arista y actualiza nodos/aristas.
- `borrarConexion()`: Usa `remove_if` con lambda para eliminar la arista por origen y destino.
- `mostrarConexiones()`: Imprime la lista de adyacencia completa en consola.
- `vaciar()`: Limpia completamente la estructura para carga fresca desde archivo.

3.3 Algoritmo de Dijkstra

El método `aplicarDijkstra()` implementa el algoritmo clásico de búsqueda de camino más corto con cola de prioridad. Puede optimizarse por tiempo o por costo según un parámetro booleano (`optimizarCosto`).

Implementación técnica:

- Usa `map<string, double>` para almacenar distancias acumuladas inicializadas en infinito.
- Usa `map<string, string>` como tabla de predecesores para reconstruir el camino.

- Usa `priority_queue` con comparador `greater<>` para obtener el nodo de menor costo primero (min-heap).
- Al finalizar, reconstruye el camino hacia atrás desde el destino hasta el origen usando la tabla de predecesores y luego lo invierte con `reverse()`.

Manejo de casos especiales:

- Verifica existencia de nodos antes de iniciar la búsqueda.
- Detecta cuando no existe ruta y muestra un mensaje descriptivo.

3.4 Persistencia de Datos (CSV / XML / JSON)

El programa implementa un sistema completo de lectura y escritura en tres formatos estándar. Las funciones de guardado iteran sobre el vector de rutas; las de apertura hacen el proceso inverso, parseando cada línea para reconstruir las conexiones.

Funciones de guardado:

- `guardarCSV()`: Escribe encabezado y filas separadas por comas.
- `guardarXML()`: Genera etiquetas anidadas por cada atributo de la ruta.
- `guardarJSON()`: Construye un arreglo de objetos con manejo correcto de comas entre elementos.

Funciones de apertura:

- `abrirCSV()`: Parsea con `stringstream` y `getline` usando ',' como delimitador.
- `abrirXML()`: Busca etiquetas de apertura y cierre para extraer valores.
- `abrirJSON()`: Usa `limpiarCadena()` para sanear claves y valores de comillas y espacios.

La función `limpiarCadena()` es una utilería que elimina espacios, tabulaciones, retornos de carro y comillas de los extremos de una cadena, siendo esencial para el parseo robusto del formato JSON.

3.5 Menú Interactivo

El programa presenta al usuario un flujo en dos etapas: primero selecciona el formato de archivo y si desea crear un árbol nuevo o cargar uno existente; luego accede al menú operativo que permite insertar, borrar, mostrar y buscar rutas, finalizando con el guardado de cambios.

4. Conclusiones

Las dos prácticas documentadas muestran una progresión clara en la complejidad y el uso de los principios de la Programación Orientada a Objetos en C++:

- La Práctica 1 establece los fundamentos del encapsulamiento, la sobrecarga y la recursividad con clases simples (Persona, Auto, Calculadora) y demuestra cómo desacoplar la lógica de presentación (reportes) de la lógica de negocio.
- La Práctica 2 eleva la complejidad hacia estructuras de datos avanzadas, integrando grafos dirigidos con pesos, algoritmos de búsqueda de ruta óptima y persistencia en formatos de intercambio estándar.
- Ambas prácticas respetan el principio de responsabilidad única: cada clase y función tiene un propósito claro y acotado.
- El uso de contenedores STL (vector, set, map, priority_queue) demuestra el aprovechamiento de la biblioteca estándar de C++ para evitar la reinención de estructuras de datos.
- El manejo de archivos con fstream y el parseo manual de formatos CSV, XML y JSON aportan experiencia práctica en interoperabilidad de datos, competencia fundamental en el desarrollo de software moderno.

En conjunto, estos programas constituyen una base sólida para proyectos de mayor escala que requieran modelado de redes, optimización de rutas y exportación de datos estructurados.